

SchemaGuard

10-Minute Video Demo Script

Presenter: Runtime: ~10 Format: Screen-recorded terminal
Pragati Narotam minutes + browser + voiceover

Pre-Demo Checklist — have ready before recording:

- Terminal at project root, virtualenv active, font size 18pt
- Browser: `http://localhost:8000/docs` (FastAPI Swagger) loaded
- Browser tab: project website `website/index.html`
- Three JSON records ready to paste (in script below)
- VS Code with `rules/healthcare_rules.py` open at HC-003
- `audit_logs/healthcare_intake_audit.jsonl` accessible

0:00 – 1:15

Segment 1 — The Hook

SCREEN: Black screen, white text: "Valid JSON. Wrong data." Hold 3 seconds → cut to terminal.

You're running an LLM pipeline that generates structured patient records or loan applications. Every single output passes JSON Schema validation — correct types, required fields, valid formats. Your pipeline is clean. Your alerts are silent.

But here's what's actually in your data.

SCREEN: Paste into terminal — large, syntax-highlighted.

```
{  
  "patient_id": "P-4412",
```

```
"admission_date": "2024-08-15",
"discharge_date": "2024-08-08", ← 7 days BEFORE
admission
"diagnosis_code": "N39.0",
"patient_age": 34
}
```

Look at the dates. Discharge is August 8th. Admission is August 15th.

This patient was discharged seven days before she was admitted. Every field has the right type. JSON Schema passes it. Without additional validation, it flows straight into your database.

SCREEN: Second terminal split — paste the finance record.

```
{
  "annual_income": 48000,
  "loan_amount": 2500000, ← 52× income
  "loan_purpose": "home_purchase"
}
```

Or this loan application. \$2.5 million on a \$48,000 salary. A 52-times loan-to-income ratio. No regulated lender would touch it. But it passes every type check — because 2,500,000 is a valid number.

These are semantic failures. The structure is correct. The logic is broken. Standard tooling has no answer for them.

SCREEN: Text overlay — "SchemaGuard" → cut to project website homepage.

1:15 – 2:30

Segment 2 — What SchemaGuard Does

SCREEN: Project website — scroll slowly through hero section, then Architecture diagram.

SchemaGuard is a semantic validation and drift detection layer for LLM-generated structured outputs. It sits between your LLM and any downstream system — database, billing pipeline, ML training set, compliance report — and catches what schema validation misses.

Every record goes through four stages.

Stage 1 — Structural validation. JSON Schema Draft 7. The baseline every team already runs.

Stage 2 — Semantic validation. Ten cross-field rules checking logical consistency, not just individual field types.

Stage 3 — Confidence scoring. A 0-to-1 score. Critical violations: -0.30 . Warnings: -0.12 . Continuous — not binary.

Stage 4 — Decision routing. Three tiers: trusted ≥ 0.85 / flagged $0.50-0.84$ / quarantined < 0.50 .

Two domains: healthcare intake, financial loan applications. Five semantic rules each. All deterministic — same input, same result, full audit trail.

2:30 – 3:30

Segment 3A — Valid Record

SCREEN: Browser — localhost:8000/docs. POST /validate → Try it out.

Let me start with the API — the same endpoint a production system would call. I'll use Swagger so you can see the full request and response.

```
POST /validate
{
  "domain": "healthcare_intake",
  "record": {
    "patient_id": "P-3021", "first_name": "James", "last_name":
"Carter",
    "date_of_birth": "1978-11-02", "admission_date": "2024-09-
14",
    "discharge_date": "2024-09-19", "diagnosis_code": "J18.9",
    "medication": "Azithromycin", "patient_age": 45, "gender":
"male",
    "insurance_provider": "Aetna", "emergency_admission": false,
    ...
  }
}
```

James Carter. 45-year-old, admitted for pneumonia, treated with Azithromycin. Every field coherent — diagnosis, medication, dates, age all line up.

SCREEN: Click Execute. Scroll to response. Highlight key fields.

```
"decision": "trusted",
"confidence_score": 1.0,
"semantic_valid": true,
"violated_rules": []
```

Structural: pass. All five semantic rules: pass. Confidence 1.0. Decision: trusted. This record flows to any downstream system without review.

3:30 – 5:00

Segment 3B — Critical Violation: HC-003

SCREEN: Clear body. Paste Sarah Mitchell record.

```
POST /validate
{
```

```

    "domain": "healthcare_intake",
    "record": {
      "patient_id": "P-4412", "first_name": "Sarah", "last_name":
      "Mitchell",
      "date_of_birth": "1990-01-20", "admission_date": "2024-08-
      15",
      "discharge_date": "2024-08-08",    ← ← ← HERE
      "diagnosis_code": "N39.0", "medication": "Ciprofloxacin",
      "patient_age": 34, ...
    }
  }
}

```

Now Sarah Mitchell. Looks completely reasonable — valid UTI diagnosis, appropriate antibiotic. But look at the dates. Discharge August 8th. Admission August 15th.

SCREEN: Execute. Highlight the violated_rules block.

```

    "decision": "flagged",
    "confidence_score": 0.70,
    "semantic_valid": false,
    "violated_rules": [{
      "rule_id": "HC-003",
      "rule_name": "discharge_after_admission",
      "severity": "critical",
      "message": "Discharge (2024-08-08) precedes admission (2024-
      08-15)"
    }]
  }
}

```

Structural: pass. Semantic: fail. Rule HC-003 fired. Discharge before admission. Critical severity. Confidence drops from 1.0 to 0.70 — that's the 0.30 critical penalty. Decision: flagged.

If this record went to a billing system, it would trigger an automatic Medicare claim rejection. If it went to an ML training set, it would teach the model that patients can leave before arriving. SchemaGuard catches it before either of those happen.

5:00 - 6:00

Segment 3C — RAG-Augmented Explanation

SCREEN: POST /rag/explain, same Sarah Mitchell record. Execute. Scroll to rag_explanation field.

Now the RAG explanation endpoint. Same record — this time, the system retrieves regulatory context from a FAISS vector store and calls Claude to generate a grounded explanation.

RAG EXPLANATION (168 words · latency: 2,842ms)

*"Under **HL7 FHIR R4** (Encounter.period), the period.end must be on or after period.start; a negative length-of-stay is mathematically impossible and will trigger automatic claim rejection by Medicare's IPPS grouper.*

*Per **CMS Medicare Claims Processing Manual Chapter 1 §30.2**, both dates should be cross-referenced against nursing admission notes, medication administration records, and any procedure log.*

Remediation: *Correct discharge_date to a date $\geq$ 2024-08-15. Do not resubmit for billing until the corrected record has been validated."*

SCREEN: Scroll to retrieved_chunks — show chunk titles and cosine scores (0.6453, 0.6109, 0.5841).

Compare that to the baseline — 41 words, naming the violation. This is 168 words. It cites HL7 FHIR R4 by resource name. It cites the CMS Manual by chapter and section. It names the downstream consequence. And it gives a specific remediation step a compliance reviewer can act on immediately.

That's the difference between telling someone *what* is wrong and telling them *why it matters* and *what to do*.

6:00 — 7:00

Segment 3D — Drift Detection

SCREEN: Terminal. Run audit log analysis.

```
$ cat audit_logs/healthcare_intake_audit.jsonl | python3 -c "
import sys, json; from collections import Counter
records = [json.loads(l) for l in sys.stdin if l.strip()]
print(f'Total records : {len(records)}')
print(f'Trusted      : {sum(1 for r in records if
r[\"decision\"]==\"trusted\")}')
print(f'Flagged      : {sum(1 for r in records if
r[\"decision\"]==\"flagged\")}')
all_rules = [v for r in records for v in
r.get(\"rules_violated\", [])]
print('Rule counts  : ', dict(Counter(all_rules)))
"

Total records : 123
Trusted      : 83
```

```
Flagged      : 40
Rule counts  : {'HC-003': 37, 'HC-001': 3, 'HC-004': 3}
```

123 real validation runs. 83 trusted, 40 flagged, zero quarantined. HC-003 appeared 37 times in this batch.

A single HC-003 violation tells me this one record has a bad date. But when HC-003 appears in 30 percent of a batch, that's a different signal — it means my LLM's date-generation behavior has shifted. Drift detection catches that population-level change by monitoring z-scores and PSI across field distributions, alerting when any signal crosses its threshold.

7:00 – 8:15

Segment 4 — Architecture

SCREEN: VS Code → rules/healthcare_rules.py → HC-003 function.

Let me show how this is built. Each rule is a Python function registered with a decorator that captures the metadata — domain, rule ID, severity, affected fields — completely decoupled from the logic.

```
@register_rule(
    domain="healthcare_intake",
    rule_id="HC-003",
    severity="critical",
    fields=["admission_date", "discharge_date"],
)
def check_discharge_after_admission(record: dict) -> RuleResult:
    passed = discharge >= admission
    return RuleResult(passed=passed, severity="critical",
        message="" if passed else
            f"Discharge ({discharge}) precedes admission
            ({admission})"
    )
```

Adding a new rule is adding a new decorated function. Adding a new domain is adding a new rules file. The pipeline never needs to change.

SCREEN: scoring/confidence.py — penalty formula. Then rag/explainer.py briefly. Then api/main.py — four include_router lines.

Confidence scoring is arithmetic — start at 1.0, subtract 0.30 per critical violation, 0.12 per warning, clamp to zero. The RAG module is a separate layer — it retrieves context from FAISS and calls Claude only when the explain endpoint is invoked. Validation itself never waits for it. And the API has four independent routers: sync validation, async batch processing, user analytics, and RAG explanations.

8:15 – 9:15

Segment 5 — Metrics

SCREEN: outputs/plots/12_summary_dashboard.png — full screen. Pause 3 seconds. Then narrate.

140 real records. 16 labeled seed records for classification metrics.

Classification

Latency

0.09ms

1.0 / 1.0 /

1.0

Precision / Recall / F1

False quarantine rate: 0%

p50 · p99 = 3.02ms

~3,800 records/second

Confidence gap

+0.24 / +0.30

HC / FN · zero overlap

RAG quality

2.7 → 6.0

Baseline → RAG (out of 6)

42 → 175 words avg

Precision, recall, F1 at 1.0 on both domains. Zero false quarantines — not a single valid record blocked, including all edge cases. Median latency 0.09 milliseconds. Explanation quality improved from 2.7 to 6.0 out of 6 across all test cases — every RAG explanation cited the applicable regulation and included a specific remediation step.

9:15 – 10:00

Segment 6 — Future Work + Close

SCREEN: Project website — Roadmap section. Highlight each card as mentioned.

Six directions to extend this. The 600-record synthetic dataset is scaffolded and ready — needs an API key and 15 minutes. The architecture is domain-agnostic, so insurance claims and e-prescriptions are two files away. Active learning would close the loop — reviewer corrections feeding back into threshold calibration. And the most interesting direction: using audit-log violation patterns to discover failure modes the system doesn't know about yet.

SCREEN: Pause on website. Cut to black.

Schema validation tells you whether JSON is well-formed. SchemaGuard tells you whether it's logically coherent. For any team using LLMs to generate structured data in production — healthcare, finance, compliance — that's the validation layer that's currently missing.

SchemaGuard

`github.com/pragatinarote/schema-guard-llm-validation`

INFO 7375 · Prompt Engineering for GenAI · Northeastern University · 2025

Timing Summary

Segment	Title	Start	End	Duration
1	The Hook (problem)	0 : 00	1 : 15	1 min 15 sec
2	What SchemaGuard does	1 : 15	2 : 30	1 min 15 sec
3A	Demo — valid record	2 : 30	3 : 30	1 min 00 sec
3B	Demo — HC-003 critical violation	3 : 30	5 : 00	1 min 30 sec
3C	Demo — RAG explanation	5 : 00	6 : 00	1 min 00 sec
3D	Demo — drift detection	6 : 00	7 : 00	1 min 00 sec
4	Architecture walkthrough	7 : 00	8 : 15	1 min 15 sec
5	Metrics highlights	8 : 15	9 : 15	1 min 00 sec
6	Future work + close	9 : 15	10 : 00	0 min 45 sec
Total				~10 : 00

Delivery Notes

- Pace:** ~140 words per minute. If running fast, add 2–3 seconds of silence after each response appears before narrating it.
- Demo segments (3A–3D):** Don't narrate while typing. Type the JSON, pause — let the viewer read the input — then speak to the output.
- On the RAG output:** Read the regulation citations out loud. "HL7 FHIR R4, Encounter.period" and "CMS Medicare Claims Processing Manual Chapter 1 Section 30.2" are the details that make the comparison compelling. Don't rush them.
- On the metrics chart:** Give the dashboard 3 full seconds of silence before speaking. Let the viewer scan it. Then guide them through it.
- Tone:** Confident, specific, no hedging. You built this — own it. Every claim in this script is backed by a real number from a real run.

